ChartNav — Consolidated Build

Backend phases 1–5: spine → state machine → auth → RBAC → CI/hardening
Repo: `thekidd2227/chartnav-platform`

Contents

1. Executive summary
2. 01 — Current state
3. 02 — Workflow state machine
4. 03 — API endpoints
5. 04 — Data model
6. 05 — Build log
7. 06 — Known gaps + verification matrix
8. 07 — Auth, org scoping, RBAC
9. 08 — Test strategy
10. 09 — CI & deploy hardening
11. 10 — Doc artifact pipeline
12. Diagram — System architecture
13. Diagram — Encounter status machine
14. Diagram — ER
15. Diagram — API / data flow

Executive summary

This document consolidates **all** ChartNav backend phases delivered to

date.

Phase 1 — Workflow spine. Alembic migration `a1b2c3d4e5f6` introduced `encounters` and `workflow_events` tables with FKs and indexes. Six encounter endpoints. Idempotent seed.

Phase 2 — State machine + filtering. `POST /encounters/{id}/status` enforces explicit forward + rework edges. `GET /encounters` accepts `organization_id`, `location_id`, `status`, `provider_name` filters.

Phase 3 — Dev auth + org scoping. `apps/api/app/auth.py` resolves the caller from `X-User-Email`. Every encounter route became org-scoped: cross-org reads return 404, cross-org body/query assertions 403.

Phase 4 — RBAC + full scoping + tests. `apps/api/app/authz.py` introduces `admin` / `clinician` / `reviewer` roles and per-edge transition rules. `/organizations`, `/locations`, `/users` are now authenticated and scoped. `{error_code, reason}` envelope is standardized. 25-test pytest suite covers auth, scoping, RBAC, state-machine invariants.

Phase 5 — CI + runtime hardening (this phase). GitHub Actions workflow runs `pip install -e ".[dev]"`, Alembic upgrade on an isolated CI DB, idempotent seed (executed twice), the full pytest suite, and a live `/health` / `/me` / `/encounters` smoke boot. A separate `docs` job regenerates the final HTML/PDF and uploads them as artifacts. A root `Makefile`, `apps/api/scripts/smoke.sh`, and `scripts/build_docs.py` codify the local verification + doc-build paths.

Preserved untouched: `/health`, `/`, Alembic history, SQLite dev workflow, state machine + filtering surface, workflow_events model.

01 — Current state

ChartNav — Current State

As of: 2026-04-17 (phase: CI + runtime hardening)

Repo layout (relevant)

```
chartnav-platform/
├── .github/workflows/
│   └── ci.yml                        # NEW — backend + docs CI
├── Makefile                          # NEW — canonical local verification
├── scripts/
│   └── build_docs.py                 # NEW — reproducible HTML/PDF builder
├── apps/
│   ├── api/
│   │   ├── app/main.py
│   │   ├── app/auth.py
│   │   ├── app/authz.py
│   │   ├── app/api/routes.py
│   │   ├── alembic.ini
│   │   ├── alembic/env.py            # honors `-x sqlalchemy.url=`
│   │   ├── alembic/versions/         # 2 migrations (unchanged)
│   │   ├── scripts_seed.py           # 2 orgs, 5 users, 3 roles
│   │   ├── scripts/smoke.sh          # NEW — curl-level smoke
│   │   ├── tests/                    # pytest suite
│   │   └── pyproject.toml            # now declares [dev] extras + pytest config
│   └── web/
├── infra/docker/docker-compose.yml
└── docs/
    ├── build/                        # living docs 01–10
    ├── diagrams/                     # Mermaid sources
    └── final/                        # generated HTML + PDF
```

Runtime baseline

- Python 3.11+, FastAPI, raw `sqlite3` driver.
- SQLite at `apps/api/chartnav.db` (gitignored). CI uses `$RUNNER_TEMP/chartnav_ci.db`.
- Alembic head: `a1b2c3d4e5f6` (no new migrations this phase).
- Auth transport: `X-User-Email` (dev). Seam: `CHARTNAV_AUTH_MODE`.
- RBAC roles: `admin`, `clinician`, `reviewer`.

- Error envelope: `{"detail": {"error_code": "...", "reason": "..."}}`.
- Every endpoint except `/health` and `/` requires auth. All data endpoints are caller-org scoped.

Verified working endpoints

Unchanged since phase 4:

- `GET /health`, `GET /` (open)
- `GET /me`
- `GET /organizations`, `GET /locations`, `GET /users` (authed + scoped)
- `GET /encounters` (+ filters)
- `GET /encounters/{id}`, `GET /encounters/{id}/events`
- `POST /encounters` (admin, clinician)
- `POST /encounters/{id}/events` (admin, clinician)
- `POST /encounters/{id}/status` (per-edge RBAC)

Automation now in place

- `make verify` — single command: reset DB, test, boot, smoke.
- `pytest tests/ -v` — 25 tests pass.
- `bash apps/api/scripts/smoke.sh <base>` — 9 curl assertions.
- `python scripts/build_docs.py` — regenerates consolidated HTML + PDF.
- `.github/workflows/ci.yml` — runs all of the above on push/PR + uploads rebuilt docs as a CI artifact.

Seeded tenants / users

org_id	slug	email	role
1	`demo-eye-clinic`	admin@chartnav.local	admin
1	`demo-eye-clinic`	clin@chartnav.local	clinician
1	`demo-eye-clinic`	rev@chartnav.local	reviewer
2	`northside-retina`	admin@northside.local	admin
2	`northside-retina`	clin@northside.local	clinician

02 — Workflow state machine

Encounter Workflow State Machine

Status values

Status	Meaning
`scheduled`	Visit is booked, patient has not arrived.
`in_progress`	Provider is seeing the patient / actively charting.
`draft_ready`	Note draft has been generated and is awaiting review.
`review_needed`	Reviewer flagged the note — needs revision or sign-off.
`completed`	Note signed off; encounter closed. **Terminal.**

Allowed transitions

Forward flow:

```
scheduled → in_progress → draft_ready → review_needed → completed
```

Explicit rework transitions (documented + enforced):

From	To	Purpose
`draft_ready`	`in_progress`	Draft rejected, return to charting.
`review_needed`	`draft_ready`	Reviewer kicks note back for rewrite.

Rejected transitions (examples that return HTTP 400)

- `scheduled → completed`
- `scheduled → review_needed`
- `scheduled → draft_ready`
- `in_progress → scheduled`
- `in_progress → completed` (must pass through draft_ready + review_needed)

- `completed → anything` (terminal)
- Any unknown status string (`invalid_status`)

Error body format:

```
{"detail":"invalid_transition: in_progress -> completed is not permitted; allowed next s
```

Idempotency

Posting the **same** status the encounter is already in is a no-op: the row

is returned unchanged and **no** `status_changed` event is appended.

Side effects on successful transition

Transition target	Timestamp action
`in_progress`	set `started_at = now()` if currently NULL
`completed`	set `completed_at = now()` ; also set `started_at = now()` if it was NULL
other	no timestamp mutation

Every successful transition appends a `workflow_events` row:

```
{
  "event_type": "status_changed",
  "event_data": {"old_status": "...", "new_status": "..."}
}
```

Diagram

See `docs/diagrams/encounter-status-machine.md` (Mermaid).

Source of truth

`apps/api/app/api/routes.py` :

- `ALLOWED_STATUSES` — the 5 permitted status strings.

- `ALLOWED_TRANSITIONS` — `dict[str, set[str]]`, one entry per source state.
- `update_encounter_status()` — the single enforcement point.

API Endpoints

Base URL (local dev): `http://127.0.0.1:8000`. All error bodies use

`{"detail": {"error_code": "...", "reason": "..."}}`.

System (open)

Method	Path
GET	`/health`
GET	`/`

Identity

Method	Path	Auth	Roles
GET	`/me`	🔒	any

Org metadata (🔒, org-scoped)

Method	Path	Behavior
GET	`/organizations`	Returns only caller's org row.
GET	`/locations`	`WHERE organization_id = caller.org`.
GET	`/users`	`WHERE organization_id = caller.org`.

Encounters (🔒, org-scoped + RBAC)

GET `/encounters`

Any authenticated role. Always `WHERE organization_id = caller.org`.

Optional query params (AND-ed, parameterized):

Param	Rule
`organization_id`	Must equal `caller.org` — else 403.
`location_id`	Narrows within caller org.
`status`	Must be in `ALLOWED_STATUSES` — else 400.
`provider_name`	Exact match.

GET `/encounters/{id}` — any role

404 if missing or cross-org.

GET `/encounters/{id}/events` — any role

404 if parent missing or cross-org.

POST `/encounters` — admin, clinician

Body:

```
{
  "organization_id": 1,
  "location_id": 1,
  "patient_identifier": "PT-1003",
  "patient_name": "Alex Chen",
  "provider_name": "Dr. Ortiz",
  "scheduled_at": "2026-04-18T14:30:00Z",
  "status": "scheduled"
}
```

POST `/encounters/{id}/events` — admin, clinician

Reviewer → 403. 404 if cross-org.

POST `/encounters/{id}/status`

Per-edge RBAC:

From	To	Roles allowed
scheduled	in_progress	admin, clinician
in_progress	draft_ready	admin, clinician
draft_ready	in_progress	admin, clinician

From	To	Roles allowed
draft_ready	review_needed	admin, reviewer
review_needed	draft_ready	admin, reviewer
review_needed	completed	admin, reviewer

Same-state POST = no-op (200). Role violation → 403 `role_cannot_transition`. Invalid transition → 400 `invalid_transition`. Cross-org → 404.

Error code inventory

Code	HTTP	Origin
`missing_auth_header`	401	auth transport
`unknown_user`	401	auth transport
`auth_mode_unsupported`	500	`CHARTNAV_AUTH_MODE` set to unknown
`role_forbidden`	403	generic `require_roles`
`role_cannot_create_encounter`	403	POST /encounters
`role_cannot_create_event`	403	POST /encounters/{id}/events
`role_cannot_transition`	403	POST /encounters/{id}/status
`cross_org_access_forbidden`	403	body/query asserts other org
`encounter_not_found`	404	missing or cross-org encounter
`location_not_found`	400	bad location_id
`invalid_status`	400	unknown status string
`invalid_initial_status`	400	POST /encounters bad init
`invalid_transition`	400	disallowed state-machine edge

Verification

This full surface is now locked in by two layers:

- pytest (`apps/api/tests/`) — 25 tests, per-test ephemeral SQLite.
- Live smoke (`apps/api/scripts/smoke.sh`) — 9 assertions against a running API.

Both run in CI on every push/PR. See `09-ci-and-deploy-hardening.md`.

Data Model

SQLite (local dev). Schema produced by Alembic migrations

`43ccbf363a8f → a1b2c3d4e5f6`. No schema changes in this RBAC/scoping phase.

Tables (unchanged this phase)

`organizations`

column	type	constraints
id	INTEGER	PK
name	VARCHAR(255)	NOT NULL
slug	VARCHAR(255)	NOT NULL, UNIQUE
created_at	DATETIME	NOT NULL default now()

`locations`

column	type	constraints
id	INTEGER	PK
organization_id	INTEGER	NOT NULL, FK → organizations(id)
name	VARCHAR	NOT NULL
created_at	DATETIME	NOT NULL default now()

`users`

column	type	constraints
id	INTEGER	PK
organization_id	INTEGER	NOT NULL, FK → organizations(id)
email	VARCHAR	NOT NULL, UNIQUE
full_name	VARCHAR	NULL

column	type	constraints
role	VARCHAR	NOT NULL default `"admin"`
created_at	DATETIME	NOT NULL default now()

Application-level role vocabulary (enforced by `app/authz.py`):

`admin`, `clinician`, `reviewer`. The column remains a free VARCHAR in

SQL; constraint is enforced at the app layer. If you need a DB-level

CHECK constraint, add it in a future migration.

`encounters` and `workflow_events`

Unchanged.

Seeded tenants (two, with full role coverage)

Org 1 — `demo-eye-clinic` (id=1)

Location: `Main Clinic` (id=1).

email	role
admin@chartnav.local	admin
clin@chartnav.local	clinician
rev@chartnav.local	reviewer

Encounters:

id	patient	provider	status	events
1	PT-1001	Dr. Carter	`in_progress`	3
2	PT-1002	Dr. Patel	`review_needed`	5

Org 2 — `northside-retina` (id=2)

Location: `Northside HQ` (id=2).

email	role
admin@northside.local	admin

email	role
clin@northside.local	clinician

Encounters:

id	patient	provider	status	events
3	PT-2001	Dr. Ahmed	`scheduled`	1

Relationships

Unchanged — see `docs/diagrams/er-diagram.md`.

Event taxonomy

event_type	Writer	Shape
`encounter_created`	`POST /encounters` / seed	`{"status": "...", "created_by": "<email>"}`
`status_changed`	`POST /encounters/{id}/status`	`{"old_status": "...", "new_status": "...", "changed_by": "<email>"}`
user-supplied	`POST /encounters/{id}/events`	Any JSON

05 — Build log

Build Log

Reverse-chronological.

2026-04-17 — Phase 5: CI + runtime hardening + doc pipeline

Step 1 — Baseline

- Starting head: `c6f29e6` (RBAC + full scoping + pytest suite).
- No migrations touched this phase.
- Tests still green locally (25/25) before any changes.

Step 2 — Packaging

- `apps/api/pyproject.toml` now declares `[project.optional-dependencies].dev = ["pytest>=8", "httpx>=0.27"]` and `[tool.pytest.ini_options]` with `testpaths = ["tests"]`. CI installs with `pip install -e "apps/api[dev]"`.

Step 3 — CI workflow

- New `.github/workflows/ci.yml` with two jobs:
- `backend` — checkout → setup-python 3.11 (pip cache) → `pip install -e ".[dev]"` → `alembic -x sqlalchemy.url=sqlite:///$RUNNER_TEMP/chartnav_ci.db upgrade head` → seed twice (idempotency proof) → `pytest tests/ -v` → boot uvicorn against the CI DB, poll `/health`, run `scripts/smoke.sh`.
- `docs` (needs: backend) — apt-install Chromium → `python scripts/build_docs.py` with `CHARTNAV_PDF_BROWSER=chromium-browser` → upload the rebuilt HTML + PDF as a `chartnav-docs-final` artifact (`if-no-files-found: error`).
- Triggers: `push` to `main` and every `pull_request`.

Step 4 — Smoke script

- New `apps/api/scripts/smoke.sh`. Shell-only curl assertions: `/health`, `/me` (401 without auth, 200 role=admin org=1 with admin header), `/encounters` (401/200), cross-org `?organization_id=2` lens → 403, own encounter 200, cross-org encounter 404. Exits non-zero on first failure.

- First draft used a helper function with an unquoted `$@` pass-through; that word-split the `X-User-Email` header and made curl hang on DNS for `admin@chartnav.local`. Rewrote to inline each assertion with proper quoting — no magic, no hangs.

Step 5 — Reproducible doc build

- New `scripts/build_docs.py` (repo-rooted — paths resolved from `__file__`). Walks `docs/build/01..10` + `docs/diagrams/*`, renders the markdown subset + Mermaid blocks, writes the consolidated HTML, then prints to PDF via headless Chromium. `CHARTNAV_PDF_BROWSER` env var lets CI point at `chromium-browser` without code changes. Reportlab fallback for a plain-text PDF if no browser is present.
- Prior script `/tmp/build_final_docs.py` was a personal scratch file — now superseded by the in-repo version.

Step 6 — Makefile (canonical local path)

- New root `Makefile` with targets: `install`, `migrate`, `seed`, `test`, `boot`, `smoke`, `docs`, `reset-db`, `clean`, and the composite `verify` (reset-db + test + boot + smoke).

Step 7 — New docs

- `docs/build/09-ci-and-deploy-hardening.md` — workflow design, local path, dev/CI DB separation, smoke contract.
- `docs/build/10-doc-artifact-pipeline.md` — builder contract, section order, fallback behavior.

Step 8 — Verification

- `make reset-db && make test` → 25/25 pytest passed (~12s).
- `make verify` → pytest green, uvicorn boot, smoke all 9 assertions green, process torn down cleanly.
- `scripts/build_docs.py` → regenerated `docs/final/chartnav-workflow-state-machine-build.{html,pdf}` deterministically.
- YAML sanity checked by loading with PyYAML; `act` not available in the shell, honest limitation documented in `06-known-gaps.md`.

Step 9 — Git hygiene

- `.pytest_cache/` and `__pycache__/` still ignored.
- No DB files committed.
- DB reset to clean seeded state before commit.

Prior phases (preserved)

Phase 4 — RBAC + full scoping + pytest suite

`authz.py`, per-edge transition roles, `/organizations` / `/locations` / `/users` now authed + org-scoped, 25-test pytest suite with per-test temp SQLite.

Phase 3 — Dev auth + org scoping

`auth.py` with `X-User-Email`, `Caller`, cross-org 404 on reads and 403 on assertions.

Phase 2 — Strict state machine + filtering

`ALLOWED_TRANSITIONS` map, 400 `invalid_transition` with allowed-next listing, `GET /encounters` filters.

Phase 1 — Workflow spine

Migration `a1b2c3d4e5f6`, idempotent seed, six encounter endpoints.

06 — Known gaps + verification matrix

Known Gaps & Verification Matrix

Verification evidence — phase 5

Automated pytest (unchanged contract, now in CI)

```
$ cd apps/api && pytest tests/ -v
... 25 passed in ~12s
```

`make verify` (canonical local gate)

Steps executed in order:

Step	Result
`rm -f apps/api/chartnav.db`	✅
`alembic upgrade head`	✅
`scripts_seed.py`	✅ (idempotent)
`pytest tests/ -v`	✅ (25/25)
`uvicorn app.main:app --port 8765` boots	✅
`/health` reachable within 10s	✅
`scripts/smoke.sh` all 9 assertions	✅
boot process torn down cleanly	✅

Smoke assertions (`scripts/smoke.sh`)

Assertion	Result
`GET /health` → 200, body `status=ok`	✅
`GET /me` without auth → 401	✅
`GET /me` with admin1 → 200, `role=admin` , `organization_id=1`	✅

Assertion	Result
`GET /encounters` without auth → 401	✅
`GET /encounters` with admin1 → 200	✅
`GET /encounters?organization_id=2` as org1 admin → 403	✅
`GET /encounters/1` as admin1 → 200	✅
`GET /encounters/3` as admin1 (cross-org) → 404	✅

Doc pipeline

- `python scripts/build_docs.py` → wrote 59KB HTML + 1.1MB PDF via headless Chrome.
- Both artifacts regenerate deterministically from `docs/build/` + `docs/diagrams/`.

CI YAML sanity

- YAML parses cleanly (verified by PyYAML load).
- Limitation: no `act` binary available to run the workflow locally; CI behavior is asserted by parity with `make verify` and by the structural review of the YAML.

Real gaps (prioritized for next phase)

1. **Auth transport still dev-only.** `X-User-Email` spoofable. Swap via `CHARTNAV_AUTH_MODE` + JWT/SSO.

2. **No `act` or local workflow runner** — YAML syntactic parse only. Add `act` or accept first-push feedback loop.

3. **No RBAC-gated write endpoints for org metadata.** `/organizations`, `/locations`, `/users` are read-only.

4. **`users.role` free VARCHAR at the DB level** — no CHECK constraint; enforced at app layer only.

5. **No pagination / cursor** on `GET /encounters`.

6. **No encounter update / delete / cancel**; no `cancelled` status.

7. **Free-form `event_data`** — no per-event_type schema.

8. **Raw `sqlite3`** per-request connections; Postgres parity (docker compose) untested.

9. **CORS `allow_origins=["*"]` ** remains.

10. **No distinct audit log** for auth/scoping failures (only successful workflow events).

11. **No rate limiting, no lockout, no structured logging.**

12. **No deploy target** — CI builds + tests but does not ship anything.

Auth, Org Scoping, RBAC

1. Authentication (authn)

Source: `apps/api/app/auth.py`.

- Transport today: request header `X-User-Email: <email>`.
- `require_caller` is a FastAPI dependency that resolves the user from

the `users` table and returns a `Caller(user_id, email, full_name, role, organization_id)`.

- Environment flag `CHARTNAV_AUTH_MODE` (default `"header"`) is the

production upgrade seam. When JWT/SSO lands, only the body of

`require_caller` (or a new branch in it) changes. Every route and

every RBAC helper continues to work unchanged because they depend on

`Caller`, not on any header.

Dev-only, explicitly

`X-User-Email` is trivially spoofable. This is acceptable for local

development only. **Do not deploy this to a shared environment

without first swapping the transport.** The seam is there; use it.

Standardized error envelope

Every auth/authz error returns:

```
{"detail": {"error_code": "<stable_code>", "reason": "<human message>"}}
```

Authn error codes

Code	HTTP	When
`missing_auth_header`	401	`X-User-Email` absent or empty.

Code	HTTP	When
`unknown_user`	401	Email not found in `users`.
`auth_mode_unsupported`	500	`CHARTNAV_AUTH_MODE` set to something unwired.

2. Authorization (authz / RBAC)

Source: `apps/api/app/authz.py`.

Roles

Role	Intent
`admin`	Full read/write inside own org.
`clinician`	Charting side of the workflow.
`reviewer`	Review side of the workflow; read-only on create/events.

All seeded users today carry exactly one of these roles.

Permission surface

Surface	admin	clinician	reviewer
Read org / locations / users	✓	✓	✓
List / read encounters	✓	✓	✓
Read encounter events	✓	✓	✓
Create encounter	✓	✓	×
Add workflow event	✓	✓	×
Transition scheduled→in_progress	✓	✓	×
Transition in_progress→draft_ready	✓	✓	×
Transition draft_ready→in_progress (rework)	✓	✓	×
Transition draft_ready→review_needed	✓	×	✓
Transition review_needed→draft_ready (kick back)	✓	×	✓
Transition review_needed→completed	✓	×	✓

Authz dependencies

- `require_roles(*roles)` — generic gate, returns `Caller`.
- `require_create_encounter` — applied to `POST /encounters`.
- `require_create_event` — applied to `POST /encounters/{id}/events`.
- `assert_can_transition(caller, from, to)` — called by the status

handler **after** the state machine accepts the edge.

Authz error codes

Code	HTTP	When
`role_forbidden`	403	Generic `require_roles` denies.
`role_cannot_create_encounter`	403	Non-admin/clinician POSTs `/encounters`.
`role_cannot_create_event`	403	Non-admin/clinician POSTs event.
`role_cannot_transition`	403	Role may not drive that specific state edge.

3. Org scoping

Caller's `organization_id` is **authoritative**. Bodies and query

strings cannot override it.

Route	Scoping
`GET /organizations`	`WHERE id = caller.org`
`GET /locations`	`WHERE organization_id = caller.org`
`GET /users`	`WHERE organization_id = caller.org`
`GET /encounters`	`WHERE organization_id = caller.org` + filters
`GET /encounters/{id}`	404 if row is cross-org
`GET /encounters/{id}/events`	404 if parent cross-org
`POST /encounters`	Body `organization_id` must equal caller's; location must belong
`POST /encounters/{id}/events`	404 if cross-org
`POST /encounters/{id}/status`	404 if cross-org

404 vs 403

- **404** when the target might-or-might-not exist in another org —

returning 403 there would leak existence.

- **403** when the client explicitly asserts a different org (body or

`?organization_id=`) — the intent is unambiguous, fail loudly.

4. Diagram

5. What this phase explicitly does NOT do

- No password auth, JWT, or SSO integration.
- No scoped writes to organizations / locations / users (meta admin).
- No audit log distinct from `workflow_events` .
- No rate limiting.
- No per-tenant key management.

These are the next auth/security phase's targets.

Test & Smoke Strategy

Stack

- `pytest` + FastAPI `TestClient` for unit/integration.
- Per-test temporary SQLite (fresh migrate + seed per test).
- Shell-level smoke via `apps/api/scripts/smoke.sh` against a real

running uvicorn.

- All of it runs in CI (`.github/workflows/ci.yml`).

Layout

```
apps/api/tests/
├── conftest.py          # fixtures: test_db, client, seeded_ids, role headers
├── test_auth.py         # authn surface
├── test_scoping.py      # org scoping across every list/read endpoint
└── test_rbac.py         # role-gated writes + state transitions

apps/api/scripts/
└── smoke.sh             # curl-level live smoke (health, me, encounters, cross-org)
```

Coverage matrix

Auth (5)

Health open; `/me` missing/empty/unknown/valid.

Scoping (8)

Orgs/locs/users scoped per tenant. `/organizations` 401 without auth.

Encounters disjoint across tenants. Cross-org GET → 404.

`?organization_id=<other>` → 403. Filters don't leak.

RBAC (12)

Admin + clinician create encounter. Reviewer create → 403. Reviewer event → 403. Clinician operational transition OK. Clinician review-stage transition 403. Reviewer complete/kick-back OK. Cross-org mutate → 404. Invalid transition preserved as 400. `status_changed` carries `old_status` / `new_status` / `changed_by`. Cross-org body mismatch → 403.

Smoke (9)

Exercise live HTTP. Commands in `09-ci-and-deploy-hardening.md`.

How to run

```
# just the pytest suite
cd apps/api && pytest tests/ -v

# everything (reset DB + pytest + boot + smoke)
make verify

# smoke only, against an already-running API
cd apps/api && bash scripts/smoke.sh http://127.0.0.1:8000
```

Results at time of writing

- `pytest`: **25 passed in ~12s**
- `make verify`: all 9 smoke assertions pass after pytest succeeds

Gaps not yet covered

- Concurrent writers / race conditions
- `event_data` schema per `event_type`
- Pagination (not yet implemented)
- Auth transport swap (needs its own harness when JWT/SSO lands)
- Long-running soak / memory behavior

CI & Deploy Hardening

GitHub Actions workflow

File: `.github/workflows/ci.yml`

Triggers: **push to `main`** and **every pull request**.

Job `backend` (ubuntu-latest, working directory `apps/api`)

1. `actions/checkout@v4`

2. `actions/setup-python@v5` — Python 3.11, pip cache on.

3. `pip install -e ".[dev]"` — installs backend plus `pytest` + `httpx` from `pyproject.toml` `[project.optional-dependencies].dev`.

4. **Migrate (isolated CI DB)** — `alembic -x "sqlalchemy.url=sqlite:///$CI_DB_PATH" upgrade head`. `CI_DB_PATH` lives under `$RUNNER_TEMP`, never in the repo.

5. **Idempotent seed run twice** — proves the seed is safe to re-run in live environments.

6. **Tests** — `pytest tests/ -v`. 25 tests must pass.

7. **API boot smoke** — launches uvicorn in-process against the CI DB, waits up to ~10s for `/health`, then runs `apps/api/scripts/smoke.sh`. Smoke script exercises `/health`, `/me` auth paths, `/encounters` scoped list, and the cross-org 403 lens.

Job `docs` (ubuntu-latest, `needs: backend`)

1. Install Chromium via apt (`chromium-browser`).

2. Run `python scripts/build_docs.py` with `CHARTNAV_PDF_BROWSER=chromium-browser`.

3. Upload `docs/final/chartnav-workflow-state-machine-build.{html,pdf}` as a CI artifact — the `docs` job fails loudly if either artifact is missing.

What CI catches

Change	Failure surface
Broken migration	`alembic upgrade` step
Broken seed	idempotent-seed step
Any pytest regression (auth, RBAC, scoping, state machine)	pytest step
App can't boot / `/health` broken	smoke script
`/me` contract drift	smoke script
Cross-org leak introduced	smoke 403 lens check
Docs pipeline regression	`docs` job

Local verification path (canonical)

One command:

```
make verify
```

Under the hood that:

- resets the dev SQLite DB,
- applies Alembic migrations,
- re-seeds idempotently,
- runs pytest,
- boots uvicorn on port 8765,
- runs `apps/api/scripts/smoke.sh`,
- kills the boot process.

Individual targets:

Target	Purpose
`make install`	venv + `pip install -e "apps/api[dev]"`
`make migrate`	Alembic upgrade head on dev DB
`make seed`	Idempotent seed on dev DB
`make test`	Full pytest suite
`make boot`	Foreground uvicorn on port 8765
`make smoke`	`scripts/smoke.sh` against running API

Target	Purpose
`make docs`	Rebuild `docs/final/*.html` + `.pdf`
`make reset-db`	`rm -f chartnav.db` + migrate + seed
`make clean`	Remove DB + caches

Dev vs CI DB separation

Surface	Path
Developer local	`apps/api/chartnav.db` (gitignored)
pytest per-test	`tmp_path / "chartnav.db"` (ephemeral)
CI	`$RUNNER_TEMP/chartnav_ci.db` (ephemeral)

The only path CI or tests ever touch inside the repo is the source code;

no DB file is shared between contexts.

Smoke script

File: `apps/api/scripts/smoke.sh`

Shell-only curl-level checks. Takes an optional `BASE_URL`:

```
scripts/smoke.sh [BASE_URL=http://127.0.0.1:8000]
```

Exits non-zero on the first failed assertion. Exercises:

- `GET /health` → 200 + `status=ok`
- `GET /me` → 401 without auth; 200 with seeded admin header; role equals `admin`
- `GET /encounters` → 401 without auth; 200 with admin header
- `GET /encounters?organization_id=2` as org1 admin → 403 (cross-org lens)
- `GET /encounters/1` as org1 admin → 200
- `GET /encounters/3` as org1 admin → 404 (cross-org)

What this phase explicitly does NOT do

- No container image build / push.

- No production deploy target.
- No Kubernetes, Terraform, or infra-as-code.
- No secrets management or signed build artifacts.

Those belong in a future "hosted deploy" phase. The seam is ready: CI

already reliably migrates, seeds, tests, and smokes the API.

Doc Artifact Pipeline

Purpose

The living docs in `docs/build/*` and `docs/diagrams/*` are the source

of truth. The final consolidated artifacts in `docs/final/` are always

rebuildable from those sources — no hand edits.

Source / output layout

```
docs/
├── build/        # human-maintained markdown — source of truth
├── diagrams/     # Mermaid sources (fenced in markdown)
└── final/
    ├── chartnav-workflow-state-machine-build.html   # generated
    └── chartnav-workflow-state-machine-build.pdf    # generated
```

Builder

File: `scripts/build_docs.py`

- Pure Python, no external deps beyond the standard library.
- Resolves paths relative to the script, so the same invocation works

locally and in CI.

- Parses a small markdown subset (headings, fenced code, tables, bullet

lists, inline `code`, **`bold`**) directly to HTML.

- Mermaid fences are passed through verbatim into `<div class="mermaid">`

blocks; the HTML imports Mermaid from a CDN and renders on load.

- Writes `docs/final/chartnav-workflow-state-machine-build.html`.
- Generates the PDF via headless Chromium (`--headless=new --print-to-pdf`).

Selection order:

1. `CHARTNAV_PDF_BROWSER` env var if set (used by CI with `chromium-browser`).

2. macOS Chrome at `/Applications/Google Chrome.app/...`.

3. Chromium in a standard app bundle.

4. `chromium-browser`, `chromium`, `google-chrome`, `chrome` on `PATH`.

- If no browser is available, falls back to `reportlab` for a plain-text

PDF so CI never silently produces a zero-byte artifact.

Run it

Locally:

```
python scripts/build_docs.py          # or:  make docs
```

In CI: the `docs` job in `.github/workflows/ci.yml` installs

`chromium-browser` via apt, runs the builder with

`CHARTNAV_PDF_BROWSER=chromium-browser`, and uploads the resulting

HTML + PDF as a `chartnav-docs-final` artifact. The upload step is

configured with `if-no-files-found: error`, so a silent generation

failure fails the build.

Sections consumed (in order)

1. Executive summary (inlined in the builder)

2. `docs/build/01-current-state.md`

3. `docs/build/02-workflow-state-machine.md`

4. `docs/build/03-api-endpoints.md`

5. `docs/build/04-data-model.md`

6. `docs/build/05-build-log.md`

7. `docs/build/06-known-gaps.md`

8. `docs/build/07-auth-and-scoping.md`
9. `docs/build/08-test-strategy.md`
10. `docs/build/09-ci-and-deploy-hardening.md`
11. `docs/build/10-doc-artifact-pipeline.md`
12. `docs/diagrams/system-architecture.md`
13. `docs/diagrams/encounter-status-machine.md`
14. `docs/diagrams/er-diagram.md`
15. `docs/diagrams/api-data-flow.md`

If a source file is missing, the builder emits an inline `*missing: <path>*` marker rather than crashing — a broken diagram or an undeleted section shows up immediately in the rendered output and in the CI artifact.

Why not a heavier stack

- No MkDocs / Docusaurus / Sphinx — those require a theme + build config we'd have to maintain. We currently just need the 10-page PDF to stay fresh.
- No pandoc dependency — keeps the CI image lean and the local experience `python` + a headless browser (which most devs already have).
- If we grow beyond a single consolidated artifact (e.g. a hosted docs site), this builder is easy to retire because the sources are already plain markdown.

Diagram — System architecture

System Architecture

```
flowchart LR subgraph Runtime Client["Client
curl · tests · apps/web"] FastAPI["FastAPI
apps/api/app/main.py"] Authn["require_caller
apps/api/app/auth.py
(CHARTNAV_AUTH_MODE seam)"] Authz["require_* / assert_can_transition
apps/api/app/authz.py"] Router["APIRouter
apps/api/app/api/routes.py"] SM["State Machine
ALLOWED_TRANSITIONS"] Scope["Org Scoping
404 cross-org read · 403 assertion"] DB[("SQLite
apps/api/chartnav.db")] end subgraph Tooling Alembic["Alembic
apps/api/alembic/"] Seed["scripts_seed.py
2 orgs · 5 users · 3 roles"] PyTest["pytest
apps/api/tests/"] Smoke["smoke.sh
apps/api/scripts/"] DocBuild["build_docs.py
scripts/"] Make["Makefile
make verify"] CI[".github/workflows/ci.yml
backend + docs jobs"] end Client -->|"HTTP + X-User-Email"| FastAPI FastAPI --> Authn Authn -->|"users lookup"| DB Authn -->|"Caller(role, org)"| Authz Authz --> Router Router --> Scope Router --> SM Router -->|"scoped SQL"| DB Alembic -.-> DB Seed -.-> DB PyTest -.->|"TestClient + temp DB"| FastAPI Smoke -.->|"curl"| FastAPI DocBuild -.->|"reads docs/build + docs/diagrams"| DocBuild Make --> Alembic Make --> Seed Make --> PyTest Make --> Smoke Make --> DocBuild CI --> Make
```

Diagram — Encounter status machine

Encounter Status State Machine

stateDiagram-v2 [*] --> scheduled [*] --> in_progress : walk-in scheduled --> in_progress in_progress --> draft_ready draft_ready --> review_needed draft_ready --> in_progress : rework review_needed --> completed review_needed --> draft_ready : kick back completed --> [*]

Legend: any edge not shown is rejected at `POST /encounters/{id}/status`

with HTTP 400 `invalid_transition`.

Diagram — ER

ER Diagram

erDiagram organizations ||--o{ locations : "has" organizations ||--o{ users : "employs (identity + role)" organizations ||--o{ encounters : "owns" locations ||--o{ encounters : "hosts" encounters ||--o{ workflow_events : "emits" organizations { int id PK string name string slug UK datetime created_at } locations { int id PK int organization_id FK string name datetime created_at } users { int id PK int organization_id FK string email UK string full_name string role "admin | clinician | reviewer" datetime created_at } encounters { int id PK int organization_id FK int location_id FK string patient_identifier string patient_name string provider_name string status datetime scheduled_at datetime started_at datetime completed_at datetime created_at } workflow_events { int id PK int encounter_id FK string event_type text event_data datetime created_at }

Seeded tenants & users

org_id	slug	email	role
1	`demo-eye-clinic`	admin@chartnav.local	admin
1	`demo-eye-clinic`	clin@chartnav.local	clinician
1	`demo-eye-clinic`	rev@chartnav.local	reviewer
2	`northside-retina`	admin@northside.local	admin
2	`northside-retina`	clin@northside.local	clinician

`users.email` is the authentication key consumed from `X-User-Email`.

`users.role` is the RBAC key consumed by `app.authz`.

`users.organization_id` is the authoritative source of scope; never

derived from client input.

Diagram — API / data flow

API / Data Flow & CI

Request path (happy + denial)

```
sequenceDiagram autonumber participant C as Client participant Authn as require_caller participant Authz as authz gate participant R as Route handler participant SM as State Machine participant DB as SQLite Note over C,DB: Happy path — clinician advances own encounter C->>Authn: POST /encounters/1/status {"status":"draft_ready"}
X-User-Email: clin@chartnav.local Authn->>DB: SELECT user DB-->>Authn: {role=clinician, org=1} Authn-->>Authz: Caller Authz->>Authz: role in {admin, clinician}? ✓ Authz-->>R: Caller R->>DB: SELECT encounter 1 DB-->>R: row(org=1, status=in_progress) R->>SM: is (in_progress → draft_ready) allowed? SM-->>R: yes R->>Authz: assert_can_transition(clinician, in_progress, draft_ready) Authz-->>R: allowed R->>DB: UPDATE + INSERT status_changed{changed_by=clin@...} R-->>C: 200 Note over C,DB: Role denied — clinician tries review-stage edge C->>Authn: POST /encounters/2/status {"status":"completed"} Authn-->>Authz: Caller(role=clinician) Authz->>SM: edge valid? SM-->>Authz: yes Authz->>Authz: clinician in role-set for (review_needed, completed)? Authz-->>C: 403 role_cannot_transition Note over C,DB: Cross-org read — 404 C->>Authn: GET /encounters/3 as clin@chartnav.local R->>DB: SELECT encounter 3 DB-->>R: row(org=2) R-->>C: 404 encounter_not_found Note over C,DB: Missing / unknown identity C->>Authn: GET /encounters (no header) → 401 missing_auth_header C->>Authn: GET /encounters (bogus email) → 401 unknown_user
```

CI gate flow

```
flowchart TD PR[["push / PR"]] subgraph backend job A[checkout] --> B[setup-python 3.11] B --> C["pip install -e \".[dev]\""] C --> D["alembic -x url=... upgrade head
(isolated CI DB)"] D --> E["seed twice (idempotency)"] E --> F["pytest tests/ -v"] F --> G["boot uvicorn + poll /health"] G --> H["scripts/smoke.sh"] end subgraph docs job I[checkout] --> J[apt chromium] J --> K["python scripts/build_docs.py"] K --> L["upload HTML + PDF artifact"] end PR --> A H --> I classDef fail fill:#fee,stroke:#c33 class F,G,H,K,L fail
```

Any failure in a red-bordered step fails CI.